



SEP2025

DRONE CHOREO "SKYORCHESTRATOR"

Software-Entwicklungspraktikum (SEP)
Sommersemester 2025

Abnahmetestspezifikation

Auftraggeber
Technische Universität Braunschweig
Institut für Robotik und Prozessinformatik
Prof. Dr. Jochen J. Steil
Mühlenpfordtstraße 23
38106 Braunschweig

Betreuer: Kilian Klankers

Auftragnehmer:

Name	E-Mail-Adresse
Mohammed Alhalabi	m.alhalabi@tu-braunschweig.de
Luan Hyseni	l.hyseni@tu-braunschweig.de
Mohamed Sabri Jlizi	m.jlizi@tu-braunschweig.de
Mohammad Khatib	m.khatib@tu-braunschweig.de
Lucas Neidahl	l.neidahl@tu-braunschweig.de
Marvin Voigt	marvin.voigt@tu-braunschweig.de

Braunschweig, 30. Mai 2025

Bearbeiterübersicht

Kapitel	Autoren	Kommentare
1	Mohamed Sabri Jlizi	fertig
2	Lucas Neidahl, Mohammad Khatib	fertig
2.1	Lucas Neidahl	fertig
2.2	Lucas Neidahl	fertig
2.3	Lucas Neidahl	fertig
2.4	Mohammad Khatib, Luan Hyseni	fertig
2.5	Mohammad Khatib	fertig
3	Mohammed Alhalabi	fertig
3.1	Mohammed Alhalabi	fertig
3.2	Mohammed Alhalabi	fertig
3.3	Lucas Neidahl	fertig
3.4	Mohammed Alhalabi	fertig
4	Lucas Neidahl	fertig

Inhaltsverzeichnis

1	Einleitung	5
2	Testplan	6
2.1	Zu testende Komponenten	6
2.2	Zu testende Funktionen/Merkmale	7
2.3	Nicht zu testende Funktionen	10
2.4	Vorgehen	10
2.4.1	Komponententests	10
2.4.2	Integrationstest	10
2.4.3	Systemtest	11
2.4.4	Abnahmetest	11
2.5	Testumgebung	11
3	Abnahmetest	13
3.1	Zu testende Anforderungen	14
3.2	Testverfahren	14
3.3	Überprüfung der Qualitätsanforderungen	15
3.3.1	Prüfung von Q10: Learnability	16
3.3.2	Prüfung von Q20: Effizienz UI	16
3.3.3	Prüfung von Q30: Fehlertoleranz UI	16
3.3.4	Prüfung von Q40: Ressourcenverbrauch	17
3.3.5	Prüfung von Q50: Dokumentation	17
3.3.6	Prüfung von Q60: Reaktionszeit UI	17
3.3.7	Prüfung von Q70: Plattformkonformität	18
3.3.8	Prüfung von Q80: Fehlertoleranz Eingaben	18
3.3.9	Testskripte	18
3.4	Testfälle	19
3.4.1	Testfall $\langle T100 \rangle$ - Szene erstellen und speichern	20
3.4.2	Testfall $\langle T200 \rangle$ - Simulation einer Drohnenshow	21
4	Glossar	24

Abbildungsverzeichnis

1 Einleitung

Im Allgemeinen ist das Testen von Software ein wesentlicher Schritt im Entwicklungsprozess, da es zur Sicherstellung der Qualität beiträgt. Mit systematischem Testen können Probleme schon frühzeitig identifiziert und gelöst werden. Auf diese Weise können Verzögerungen im Projektverlauf vermieden und die im Pflichtenheft festgelegten Anforderungen zuverlässig umgesetzt werden. Vor allem bei Projekten mit komplizierten Nutzerinteraktionen und grafischen Simulationen – wie bei SkyOrchestrator – ist es essenziell, von Anfang an und fortlaufend Tests durchzuführen, um eine stabile und benutzerfreundliche Anwendung sicherzustellen. Diese Abnahmetestspezifikation betrifft die Software SkyOrchestrator, die im Sommersemester 2025 im Rahmen eines Softwareentwicklungspraktikums an der Technischen Universität Braunschweig entwickelt wurde. Das Ziel ist, eine benutzerfreundliche Plattform zur Planung, Simulation und Visualisierung von Drohnenshows zu entwickeln. Ziel ist es, dass die Software sich an Kreative ohne Programmierkenntnisse sowie an Profi-Eventplaner richtet, damit diese auf einfache Weise komplexe Drohnenchoreografien entwerfen und simulieren können. Die Tests richten sich nach dem IEEE 829-Standard für Software-Testdokumente. Dieser Standard definiert eine klare Struktur für Testberichte und ermöglicht so eine nachvollziehbare Qualitätssicherung. SkyOrchestrator muss bestimmte Kriterien erfüllen, insbesondere in Bezug auf Funktionen, Benutzerfreundlichkeit und Erweiterbarkeit. Es soll eine flüssige 3D-Simulation basierend auf den Eingaben der Nutzer bieten, eine intuitive Benutzeroberfläche haben und visuelle Formationen sowie Lichteffekte präzise darstellen. Die Technologie nutzt Python 3.13 und die Simulationsbibliothek PyBullet, wobei Windows 11 die Zielplattform ist. Bei den Abnahmetests soll kontrolliert werden, ob die Software alle definierten Anforderungen erfüllt. Typische Nutzungsszenarien, Funktionsaufrufe und das Systemverhalten unter unterschiedlichen Bedingungen werden getestet. Die Software wird erst dann als fehlerfrei und bereit für die Abnahme durch den Auftraggeber angesehen, wenn alle Tests bestanden wurden.

2 Testplan

Im folgenden Kapitel wird der Testplan für den Skyorchester beschrieben. Dabei wird sowohl auf die Komponenten und Funktionen eingegangen, die getestet werden sollen, als auch auf solche, die nicht getestet werden. Außerdem wird die allgemeine Vorgehensweise zum Testen der Funktionen beschrieben und erklärt.

2.1 Zu testende Komponenten

Die folgenden Hauptkomponenten unserer Software werden getestet:

- **GUI** Benutzeroberfläche zur Nutzerinteraktion mit der Software. Der Nutzer kann hier die Drohnenshow konfigurieren, simulieren und Sie verwalten. Hierbei muss getestet werden:
 - Richtige Darstellung und intuitive Bedienung ohne Programmierkenntnisse
 - Fehlerfreie Eingabeverarbeitung und visuelle Rückmeldungen.
 - Funktionalität der Timeline-Komponente.
- **Show-Konfigurationsmodul** Es verarbeitet die Nutzereingaben zur Erstellung und Verwaltung der Show-Logik. Hierbei muss getestet werden:
 - Übernahme, Validierung und Speicherung von Show-Parametern.
 - Konsistente Verwaltung von Szenen und Formationswechseln.
 - Vollständige Datenaufbereitung für die Simulation
- **Simulations-Engine** Berechnet Drohnenbewegungen, Lichteffekte und stellt die 3D-Umgebung dar. Hierbei muss getestet werden:
 - Korrekte Umsetzung der Choreographie-Parameter und Lichteffekte
 - Performance und Skalierbarkeit
- **Projektmanagement-Modul** Zuständig für das Speichern und Laden von Drohnenshow-Projekten. Hierbei muss getestet werden:

- Fehlerfreies Speichern und Laden von Projekten.
- Datenintegrität nach Speichern und Laden.
- Korrekter Umgang mit fehlerhaften Projektdateien.

2.2 Zu testende Funktionen/Merkmale

Hier geht es um alle Funktionen und Eigenschaften, die wir uns beim Testen genau ansehen wollen. Wir listen hier alles auf, was getestet werden soll und beziehen uns dabei auf die Funktions-IDs aus dem Pflichtenheft. Außerdem werden auch hier die Messkriterien getestet. Die Software SkyOrchestrator wird als quelloffenes Python-Projekt über ein zentrales GitLab-Repository bereitgestellt. Die Anwendung liegt am Ende als .exe Datei vor. Die Anwendung wird sowohl in der .exe Form als auch in Form von Quellcode, Skripten und Konfigurationsdateien vom Git-Repository getestet.

Der Zugriff auf das Git Repository benötigt eine Internetverbindung und Speicherplatz für das Klonen des Repositories. Die Installation der Python-Umgebung und der Abhängigkeiten benötigt ebenfalls etwa 500MB Speicherplatz.

Vor der Durchführung der Tests muss die Software wie folgt vorbereitet werden:

- Klonen des aktuellen Entwicklungsstandes aus dem GitLab-Repository.
- Einrichtung einer Python 3.13 Umgebung (z.B. mittels venv).
- Installation aller erforderlichen Abhängigkeiten, die in einer requirements.txt-Datei im Repository spezifiziert sind (insbesondere PyBullet und die für Tkinter benötigten Systembibliotheken, falls nicht bereits vorhanden). Dies erfolgt typischerweise mit dem Befehl `pip install -r requirements.txt` innerhalb der aktivierten virtuellen Umgebung.
- Die Software wird direkt aus dem Quellcode durch Ausführen des Hauptskripts gestartet. Eine separate Kompilierung ist nicht notwendig."

Folgende Hauptfunktionen der Anwendung werden getestet:

(F 10) Konfiguration der Show

- Der Benutzer plant die Inhalte einer Drohnenshow durch Erstellung, Bearbeitung und Verwaltung einzelner Szenen. Die Drohnenshow besteht aus den vorher konfigurierten Szenen mit definierter Bewegung, Beleuchtung und Zeitsteuerung, die zur Simulation übergeben werden können. Alle grundlegenden Planungsfunktionen müssen so gestaltet sein, dass die Bedienung ohne Programmierkenntnisse möglich ist (Testaspekt für RM4).

- Zugehörige Muss-Kriterien für die Gesamtfunktion $\langle F\ 10 \rangle$: $\langle RM1 \rangle$, $\langle RM2 \rangle$, $\langle RM3 \rangle$, $\langle RM4 \rangle$, $\langle RM5 \rangle$, $\langle RM6 \rangle$, $\langle RM7 \rangle$.
- Die Konfiguration der Show (F10) unterteilt sich in folgende spezifische Funktionalitäten, die detailliert getestet werden:
 - **$\langle F11 \rangle$ Szene anlegen und Metadaten erfassen**
 - **$\langle F12 \rangle$ Formation und Bewegung definieren**
 - **$\langle F13 \rangle$ Licht- und Zeiteffekte konfigurieren**
 - **$\langle F14 \rangle$ Szene speichern und zur Simulation übergeben**

⟨F 20⟩ Choreographie simulieren

- Eine zuvor konfigurierte Drohnenshow wird in der Simulation abgespielt. Die geplante Choreographie wird in einer 3D-Umgebung möglichst flüssig und in Echtzeit simuliert.
- Zugehörige Muss-Kriterien: ⟨RM1⟩, ⟨RM4⟩, ⟨RM5⟩

⟨F 30⟩ Projekt verwalten

- Ein Projekt wird gespeichert, geladen oder exportiert. Der Benutzer kann eine bestehende Konfiguration speichern, wiederherstellen oder in ein Austauschformat exportieren.
- Zugehörige Muss-Kriterien: ⟨RM4⟩, ⟨RM5⟩

⟨F 40⟩ Szenenprüfung auf Eingabefehler

- Der Benutzer überprüft eine Szene vor dem Speichern oder der Simulation. Das System erkennt fehlende Pflichtangaben oder ungültige Werte und zeigt sie gesammelt an. Fehlerhafte oder unvollständige Eingaben sollen dem Benutzer klar gemeldet werden, damit diese korrigiert werden können.

Zusätzlich zu den funktionalen Anforderungen werden auch die folgenden Qualitätsanforderungen (gemäß Pflichtenheft Kapitel 6.5) im Rahmen der Tests überprüft, um die allgemeine Güte und Gebrauchstauglichkeit der Software sicherzustellen:

- **⟨Q10⟩ Learnability:** Beherrschbarkeit der Grundfunktionen binnen 15 Minuten ohne Anleitung.
- **⟨Q20⟩ Effizienz:** Erreichbarkeit häufig genutzter Funktionen mit maximal zwei Klicks.
- **⟨Q30⟩ Fehlertoleranz (UI):** Unterstützung bei Eingabefehlern durch Validierung, Vorschläge, Korrekturhinweise.
- **⟨Q40⟩ Ressourcenverbrauch:** Speicherverbrauch unter 1 GB RAM bei Simulation mit 50 Drohnen.
- **⟨Q50⟩ Dokumentation:** Verfügbarkeit kontextsensitiver Hilfetexte und Tooltips für Hauptfunktionen.
- **⟨Q60⟩ Reaktionszeit UI:** Reaktionen auf Benutzeraktionen (außerhalb Simulation) unter 1 Sekunde.
- **⟨Q70⟩ Plattformkonformität:** Lauffähigkeit auf Python 3.11+ unter Windows 10/11 ohne Codeänderungen.
- **⟨Q80⟩ Fehlertoleranz (Eingaben):** Erkennung ungültiger Benutzereingaben und verständliche Fehlermeldung innerhalb 1 Sekunde.

2.3 Nicht zu testende Funktionen

Alle in dem Kapitel Produktfunktionen beschriebenen Funktionalitäten aus dem Pflichtenheft ($\langle F 10 \rangle$, $\langle F 11 \rangle$, $\langle F 12 \rangle$, $\langle F 13 \rangle$, $\langle F 20 \rangle$, $\langle F 30 \rangle$, $\langle F 40 \rangle$) werden von uns getestet. Außerdem werden die Qualitätsanforderungen Q10, Q20, Q30, Q40, Q50, Q60, Q70, Q80 ebenfalls überprüft.

2.4 Vorgehen

In diesem Abschnitt wird das grundsätzliche Vorgehen zur Durchführung der Softwaretests erläutert. Ziel ist es, sicherzustellen, dass alle wesentlichen Funktionen von SkyOrchestrator zuverlässig und korrekt arbeiten. Die Testmethodik orientiert sich am V-Modell und unterscheidet vier aufeinander aufbauende Teststufen:

2.4.1 Komponententests

Die in Kapitel 2.1 beschriebenen Hauptkomponenten von SkyOrchestrator werden in dieser Teststufe einzeln und unabhängig voneinander getestet. Dabei werden gezielt einzelne Module isoliert geprüft. Die Tests erfolgen automatisiert mit dem Python-Framework pytest. Für jede Funktionseinheit werden definierte Testfälle implementiert, um sowohl korrekte als auch fehlerhafte Eingaben zu überprüfen. Die erwarteten Ausgaben werden mit den tatsächlichen Ergebnissen verglichen und systematisch protokolliert.

2.4.2 Integrationstest

Nach Abschluss der Unit-Tests werden die einzelnen Komponenten von SkyOrchestrator schrittweise zu größeren Funktionseinheiten zusammengeführt. Dabei wird das Bottom-Up-Prinzip angewendet: Zunächst werden kleinere Module miteinander verbunden, anschließend folgen Tests auf höheren Ebenen, etwa zwischen Konfigurationslogik, Simulationsmodul und Projektverwaltung. Die Tests werden auf Basis definierter Anwendungsfälle durchgeführt und die Ergebnisse schriftlich protokolliert. Durch die frühzeitige Überprüfung von Schnittstellen und Modulverbindungen können Probleme in späteren Testphasen vermieden werden.

2.4.3 Systemtest

Nach erfolgreichem Abschluss der Integrationstests wird das Gesamtsystem SkyOrchestrator als vollständige Anwendung getestet. Ziel ist es, sowohl funktionale als auch nicht-funktionale Anforderungen unter realitätsnahen Bedingungen zu prüfen. Die Tests finden in einer Testumgebung statt, die der geplanten Zielumgebung beim Nutzer möglichst genau entspricht.

Im Mittelpunkt stehen hier Black-Box-Tests: Das Systemverhalten wird ausschließlich aus Sicht der Anwender:innen geprüft, ohne auf interne Implementierungsdetails einzugehen. Alle Hauptfunktionen – von der Show-Konfiguration über die Simulation bis zur Projektverwaltung – werden getestet. Zusätzlich wird auf Aspekte wie Benutzerfreundlichkeit, Reaktionszeit und Stabilität geachtet.

Die Testfälle basieren auf den Anforderungen aus dem Pflichtenheft. Die Ergebnisse werden dokumentiert und bei Fehlern durch Screenshots, Logdateien oder direkte Beobachtungen ergänzt. Der Systemtest stellt damit sicher, dass die Software im geplanten Nutzungskontext zuverlässig funktioniert und für den Kunden einsatzbereit ist.

2.4.4 Abnahmetest

Nach Abschluss aller vorangegangenen Teststufen wird der Abnahmetest durchgeführt. Ziel ist es, die Produktreife aus Sicht des Auftraggebers zu bestätigen und die im Pflichtenheft definierten Anforderungen vollständig zu überprüfen. Die Tests werden in einer produktionsnahen Umgebung durchgeführt und orientieren sich strikt an den funktionalen Muss-Kriterien. Der Abnahmetest dient nicht mehr primär der Fehlersuche, sondern der formellen Bestätigung, dass die Software vollständig und vertragsgemäß geliefert wurde. Dafür wird SkyOrchestrator mithilfe typischer Anwendungsszenarien getestet, beispielsweise durch das Anlegen, Speichern und Simulieren komplexer Shows.

2.5 Testumgebung

Als Testumgebung kommt ein lokal eingerichtetes Testsystem unter Windows 11 zum Einsatz, das der Zielpattform der Software entspricht. Für die automatisierten Komponententests wird die Python-Testbibliothek pytest verwendet. Die Tests laufen auf einem leistungsfähigen Desktop-PC mit folgender Ausstattung:

- Intel Core i7-7700K CPU
- 32 GB RAM

- NVIDIA RTX 2080 GPU
- SSD-Speicher

Zur Validierung exportierter Daten werden JSON-Validatoren und Texteditoren wie VS Code und Notepad++ verwendet. Für die Fehleranalyse werden zusätzlich Logdateien sowie Bildschirmaufzeichnungen herangezogen. Diese Umgebung ermöglicht es, sowohl automatisierte Unit-Tests als auch realitätsnahe manuelle Blackbox-Tests unter produktionsnahen Bedingungen durchzuführen.

3 Abnahmetest

In diesem Kapitel werden die funktionalen Anforderungen der Software SkyOrchestrator anhand festgelegter Testfälle überprüft. Die im Pflichtenheft festgelegten Muss-Kriterien und Kernfunktionen bilden die Grundlage für die Tests, deren Ziel es ist, dem Auftraggeber eine Entscheidung über die Abnahme der Software zu ermöglichen.

Es werden typische Nutzungsszenarien aus der Perspektive der Endnutzer simuliert. Die Auswertung basiert auf den in Abschnitt 3.3 Testfälle beschriebenen Einzelschritten, erwarteten Resultaten und Kriterien zur Bewertung der Testergebnisse.

Die Tests werden manuell auf einem Standard-Testsystem mit Windows 10 durchgeführt. Jeder Testfall wird separat dokumentiert und mit dem Ergebnis (bestanden/nicht bestanden) versehen. Im Abnahmeprotokoll werden Abweichungen, Fehler oder unerwartete Systemverhalten festgehalten.

Für die Systemabnahme durch den Auftraggeber ist es notwendig, dass alle Muss-Testfälle erfolgreich durchgeführt wurden.

3.1 Zu testende Anforderungen

Im Rahmen der Abnahmetests werden die folgenden funktionalen Anforderungen überprüft. Sie sind essenziell für die Funktionalität der Anwendung und wurden aus dem Pflichtenheft abgeleitet:

Nr	Anforderung	Testfälle	Kommentar
1	<Anlegen und Konfigurieren einer Szene (<F10>, <F11>, <F12>, <F13>, <F14>)>	<T100>	Basisfunktion zur Erstellung und Vorbereitung einer Show
2	<Starten und Beobachten der Simulation (<F20>)>	<T200>	Visualisierung und Ablaufprüfung der choreografierten Szenen
3	<Speichern und Laden von Projekten (<F30>)>	<T300>	Sicherstellung der Datenpersistenz
4	<Automatische Erkennung von Eingabefehlern in Szenen (<F40>)>	<T400>	Erhöht Bedienkomfort und reduziert Fehler beim Speichern oder Simulieren

3.2 Testverfahren

Die Abnahmetests für SkyOrchestrator erfolgen durch manuelle, dynamische Black-Box-Tests. Der Schwerpunkt liegt auf der Überprüfung der Funktionen, die für den Benutzer sichtbar sind, ohne Berücksichtigung des Quellcodes oder interner Programmstrukturen.

Die Testfälle basieren auf typischen Nutzungssituationen des Endanwenders. Die Ausführung findet auf einem Standard-Testsystem mit Windows 11 sta. In der finalen GUI der Anwendung wird getestet.

Bei jedem Testfall werden die folgenden Aspekte kontrolliert:

1. Richtigkeit der Ein- und Ausgaben
2. Antwort auf falsche Eingaben
3. Visuelles Nutzerverhalten im Interface
4. Logdatei-Einträge

5. Verhalten des Systems bei Speicherung und Simulation

Die Bewertung erfolgt nach dem „Bestanden / Nicht bestanden“-Prinzip, das sich auf klar definierte Kriterien im jeweiligen Testfall stützt.

3.3 Überprüfung der Qualitätsanforderungen

Darüber hinaus werden die folgenden Qualitätsanforderungen (gemäß Pflichtenheft Kapitel 6.5) überprüft.

ID	Anforderung (Qualität)	Prüfverfahren		Kommentar
Q10	Learnability (15 Min. Einarbeitung)	Siehe	Abschn. 3.3.1	Grundlegende Bedienbarkeit
Q20	Effizienz UI (max. 2 Klicks)	Siehe	Abschn. 3.3.2	Schnelle Erreichbarkeit von Kernfunktionen
Q30	Fehlertoleranz UI (Validierung, Hinweise)	Siehe	Abschn. 3.3.3	Nutzerunterstützung bei Fehleingaben
Q40	Ressourcenverbrauch (max. 1GB RAM bei 50 Drohnen)	Siehe	Abschn. 3.3.4	Effiziente Nutzung von Systemressourcen
Q50	Dokumentation (Hilfetexte, Tooltips)	Siehe	Abschn. 3.3.5	Unterstützung direkt in der Anwendung
Q60	Reaktionszeit UI (max. 1 Sek. außerhalb Simulation)	Siehe	Abschn. 3.3.6	Flüssige Bedienung der Oberfläche
Q70	Plattformkonformität (Python 3.11+, Win10/11)	Siehe	Abschn. 3.3.7	Lauffähigkeit auf Zielsystemen
Q80	Fehlertoleranz Eingaben (1 Sek. Meldung)	Siehe	Abschn. 3.3.8	Schnelle Rückmeldung bei spezifischen Fehleingaben

Neben den funktionalen Testfällen (T100-T400) werden die im Pflichtenheft spezifizierten Qualitätsanforderungen (Q10-Q80) wie folgt überprüft:

3.3.1 Prüfung von Q10: Learnability

- **Anforderung:** Neue Nutzer:innen sollen die Grundfunktionen (Choreografie erstellen, Simulation starten) binnen 15 Minuten ohne Anleitung beherrschen.
- **Testmethode:** Ein Probanden-Test wird mit mindestens einer Person durchgeführt, die die Software erstmalig verwendet. Die Testperson erhält die Aufgabe, eine einfache Szene zu erstellen und zu simulieren. Die benötigte Zeit und eventuelle Schwierigkeiten werden protokolliert.
- **Erfolgskriterium:** Die Aufgabe wird innerhalb von 15 Minuten erfolgreich und ohne externe Hilfe abgeschlossen.

3.3.2 Prüfung von Q20: Effizienz UI

- **Anforderung:** Häufig genutzte Funktionen (z.B. Drohnen hinzufügen, Formation wechseln) müssen mit maximal zwei Klicks erreichbar sein.
- **Testmethode:** Für eine vordefinierte Liste häufig genutzter Funktionen wird der Klickpfad vom jeweiligen Ausgangspunkt in der GUI analysiert und die Anzahl der notwendigen Klicks gezählt.
- **Erfolgskriterium:** Keine der getesteten häufig genutzten Funktionen erfordert mehr als zwei Klicks.

3.3.3 Prüfung von Q30: Fehlertoleranz UI

- **Anforderung:** Die UI unterstützt Eingabefehler durch automatische Validierung, Vorschläge und Korrekturhinweise.
- **Testmethode:** In verschiedenen Eingabefeldern der Benutzeroberfläche (z.B. für Szenenparameter, Drohnenkonfiguration) werden gezielt fehlerhafte oder unvollständige Eingaben getätigt (Text statt Zahl, Werte außerhalb des Gültigkeitsbereichs, leere Pflichtfelder).
- **Erfolgskriterium:** Das System validiert die Eingaben, verhindert die Übernahme fehlerhafter Daten und gibt klare, verständliche Hinweise oder Fehlermeldungen zur Korrektur aus.

3.3.4 Prüfung von Q40: Ressourcenverbrauch

- **Anforderung:** Der Speicherverbrauch der Anwendung darf während einer Simulation mit 50 Drohnen 1 GB RAM nicht überschreiten.
- **Testmethode:** Eine Testszene mit 50 Drohnen und repräsentativen Flugmanövern wird für mindestens 2 Minuten simuliert. Der maximale RAM-Verbrauch des Anwendungsprozesses wird währenddessen mit Systemwerkzeugen (z.B. Task-Manager unter Windows) gemessen.
- **Erfolgskriterium:** Der gemessene RAM-Spitzenwert während der Simulation bleibt unter 1 GB.

3.3.5 Prüfung von Q50: Dokumentation

- **Anforderung:** Kontextsensitive Hilfetexte und Tooltips sind für alle Hauptfunktionen verfügbar.
- **Testmethode:** Die Benutzeroberfläche wird systematisch auf das Vorhandensein und die Qualität von Tooltips für alle interaktiven Elemente der Hauptfunktionen (Buttons, Regler, wichtige Eingabefelder) überprüft. Falls kontextsensitive Hilfetexte über andere Mechanismen bereitgestellt werden, werden diese ebenfalls geprüft.
- **Erfolgskriterium:** Für alle wesentlichen Bedienelemente der Hauptfunktionen sind informative Tooltips oder äquivalente Hilfestellungen vorhanden.

3.3.6 Prüfung von Q60: Reaktionszeit UI

- **Anforderung:** Alle Reaktionen auf Benutzeraktionen außerhalb der Simulation (z.B. Szenen anlegen, Parameter ändern) dürfen 1 Sekunde nicht überschreiten.
- **Testmethode:** Eine Reihe typischer Benutzerinteraktionen außerhalb der laufenden Simulation (z.B. Öffnen von Dialogen, Anlegen einer neuen Szene, Ändern von Parametern in Formularen) wird durchgeführt. Die Zeit von der Aktion des Nutzers bis zur Reaktion des UI wird subjektiv bewertet bzw. stichprobenartig mit einer Stoppuhr gemessen.
- **Erfolgskriterium:** Alle getesteten UI-Reaktionen erfolgen innerhalb von 1 Sekunde.

3.3.7 Prüfung von Q70: Plattformkonformität

- **Anforderung:** Die Software muss ohne Codeänderungen auf Python 3.11+ unter Windows 10/11 starten und lauffähig sein.
- **Testmethode:** Die Software wird auf Testsystemen mit Windows 10 (64 Bit) und Windows 11 (64 Bit), jeweils mit einer kompatiblen Python-Version (z.B. 3.13 gemäß Pflichtenheft), gemäß Installationsanleitung eingerichtet und gestartet. Basale Funktionen (z.B. Projekt öffnen, Szene anlegen, kurze Simulation) werden ausgeführt.
- **Erfolgskriterium:** Die Software ist auf beiden Plattformen ohne Code-Modifikation installierbar, startbar und grundlegend funktionsfähig.

3.3.8 Prüfung von Q80: Fehlertoleranz Eingaben

- **Anforderung:** Ungültige Benutzereingaben (z.B. fehlerhafte Koordinaten) müssen durch Validierungsmechanismen erkannt werden, und eine verständliche Fehlermeldung innerhalb von 1 Sekunde angezeigt werden.
- **Testmethode:** Gezielte Fehleingaben für spezifische Datentypen (z.B. Koordinaten mit Buchstaben, ungültige Zeitformate, Werte außerhalb definierter Grenzen) werden in relevanten Eingabefeldern vorgenommen. Es wird geprüft, ob eine Validierung stattfindet und wie schnell eine verständliche Fehlermeldung erscheint.
- **Erfolgskriterium:** Ungültige Eingaben werden korrekt erkannt und eine verständliche Fehlermeldung wird dem Nutzer innerhalb von 1 Sekunde nach der Eingabe oder dem Versuch der Übernahme angezeigt.

3.3.9 Testskripte

Bei der Durchführung der Abnahmetests kommen keine automatisierten Testskripte zum Einsatz. Die Durchführung aller Testfälle erfolgt manuell, basierend auf den in Abschnitt 3.3 erläuterten Einzelschritten.

Die Testergebnisse werden aufgezeichnet, indem die Benutzeroberfläche visuell kontrolliert, das Verhalten des Systems beobachtet und Logdateien geprüft werden.

Sollte es während eines Tests zu einem Fehler kommen, wird dieser nachvollziehbar im Testprotokoll festgehalten. Eine strukturierte Erfassung erfolgt in tabellarischer Form zur Nachverfolgbarkeit.

3.4 Testfälle

Im Folgenden werden die Abnahmetestfälle für das System **SkyOrchestrator** beschrieben. Jeder Testfall bezieht sich auf eine im Pflichtenheft festgelegte Funktion und prüft, ob diese unter realistischen Bedingungen korrekt ausgeführt wird.

Die Testfälle sind nummeriert ($\langle T100 \rangle$ bis $\langle T400 \rangle$) und enthalten folgende Angaben:

- Ziel des Tests
- Betroffene Funktionen
- Pass/Fail-Kriterien
- Vorbedingungen
- Einzelschritte und erwartete Ausgabe
- Beobachtungen, Besonderheiten und Abhängigkeiten

3.4.1 Testfall $\langle T100 \rangle$ - Szene erstellen und speichern

Ziel

Überprüfung, ob ein neue Szene mit allen notwendigen Metadaten erstellt, konfiguriert und erfolgreich gespeichert werden kann.

Objekte/Methoden/Funktionen

$\langle F10 \rangle$ Show konfigurieren, $\langle F11 \rangle$ Szene anlegen, $\langle F12 \rangle$ Formation definieren, $\langle F13 \rangle$ Licht-/Zeiteffekte, $\langle F14 \rangle$ Szene speichern; RM1, RM2, RM4, RM6

Pass/Fail Kriterien

Bestanden: Die Szene wird erfolgreich gespeichert und erscheint korrekt in der Szenenübersicht

Nicht bestanden: Die Szene wird nicht gespeichert, oder es erscheint eine Fehlermeldung bei gültigen Eingaben.

Vorbedingung

Die Software ist gestartet.

Ein Projekt wurde neu erstellt oder geöffnet.

Einzelschritte:

1. In der Anwendung auf „Neue Szene erstellen“ klicken
2. Szenenname, Dauer und Anzahl der Drohnen eingeben
3. Formation und Bewegungsmuster auswählen
4. Auf „Szene speichern“ klicken

Ausgabe:

Die Szene erscheint in der Szenenliste mit allen eingegebenen Parametern.

Beobachtungen / Log / Umgebung

Die GUI zeigt die Szene korrekt an.

In der Logdatei steht z.B.: Szene erfolgreich gespeichert.

Besonderheiten

Gültige Eingaben erforderlich (z.B. keine leeren Felder).

Abhängigkeiten

Keine

3.4.2 Testfall $\langle T200 \rangle$ - Simulation einer Drohnenshow

Ziel

Überprüfung, ob die gespeicherte Szene korrekt in der 3D-Umgebung simuliert wird.

Objekte/Methoden/Funktionen

$\langle F20 \rangle$ Simulation starten RM3, RM6, RS1

Pass/Fail Kriterien

Bestanden: Die Simulation startet fehlerfrei, läuft flüssig ab und zeigt alle konfigurierten Bewegungen und Effekte.

Nicht bestanden: Die Simulation bleibt hängen, zeigt falsche Abläufe oder gibt Fehler aus.

Vorbedingung

Mindestens eine vollständige Szene ist gespeichert.

Einzelschritte

1. Szene aus der Übersicht auswählen.

2. „Simulation starten“ klicken.

Ausgabe: Die Szene wird in Echtzeit visualisiert, inklusive Drohnenbewegungen und Lichtanimationen.

Beobachtungen / Log / Umgebung

3D-Simulation läuft sichtbar.

Logeintrag: Simulation erfolgreich ausgeführt.

Besonderheiten

Simulation sollte auf einem System mit ausreichender Grafikleistung getestet werden.

Abhängigkeiten

Abhängig von Testfall $\langle T100 \rangle$

3.3.3 Testfall <T300> – Projekt speichern und laden

Ziel:

Überprüfung, ob ein Projekt korrekt gespeichert und anschließend fehlerfrei wieder geladen werden kann.

Objekte/Methoden/Funktionen:

<F30> Projekt speichern und laden; RS3,RM4,RM5

Pass/Fail Kriterien:

- **Bestanden:** Das Projekt wird vollständig gespeichert und kann nach dem Neustart der Anwendung ohne Datenverlust geladen werden.
- **Nicht bestanden:** Projektdatei wird nicht erstellt, beim Laden treten Fehler auf oder Daten (z.B. Szenen) fehlen.

Vorbedingung:

Ein Projekt mit mindestens einer vollständig konfigurierten Szene ist geöffnet.

Einzelschritte:

1. Auf „Projekt speichern“ klicken
2. Einen Dateinamen eingeben (z.B. `abendshow1`)
3. Anwendung schließen
4. Anwendung neu starten
5. Über „Projekt laden“ die zuvor gespeicherte Datei auswählen

Ausgabe:

Nach dem Laden erscheinen alle Szenen und Konfigurationen im gleichen Zustand wie vor dem Speichern.

Beobachtungen / Log / Umgebung:

Die GUI zeigt die gespeicherten Szenen korrekt an. In der Logdatei steht z.B.: `Projekt erfolgreich geladen`.

Besonderheiten:

Test kann sowohl mit manuellem Speichern als auch mit automatischer Speicherfunktion (falls vorhanden) wiederholt werden.

Abhängigkeiten:

Abhängig von Testfall <T100>

3.3.4 Testfall <T400> – Automatische Erkennung von Eingabefehlern

Ziel:

Überprüfung, ob das System ungültige oder unvollständige Eingaben beim Anlegen oder Speichern einer Szene korrekt erkennt und eine verständliche Fehlermeldung ausgibt.

Objekte/Methoden/Funktionen:

<F40> Eingabevalidierung RM1, RM4, RM7

Pass/Fail Kriterien:

- **Bestanden:** Das System erkennt die fehlerhafte Eingabe und verhindert das Speichern. Eine verständliche Fehlermeldung wird angezeigt.
- **Nicht bestanden:** Die Szene wird trotz falscher oder unvollständiger Angaben gespeichert, oder es erfolgt keine Rückmeldung an den Benutzer.

Vorbedingung:

Der Benutzer befindet sich im Formular zum Anlegen einer neuen Szene.

Einzelschritte:

1. Auf „Neue Szene erstellen“ klicken
2. Szenenname eingeben, aber Dauer oder Anzahl Drohnen leer lassen
3. Auf „Szene speichern“ klicken

Ausgabe:

Das System zeigt eine Fehlermeldung wie: "Bitte alle Pflichtfelder ausfüllen." Die Szene wird nicht gespeichert.

Beobachtungen / Log / Umgebung:

In der GUI erscheint eine Warnung. In der Logdatei steht z.B.: **Speichern abgebrochen - Eingabefehler.**

Besonderheiten:

Pflichtfelder (z.B. Szenenname, Dauer, Drohnenanzahl) müssen vollständig und plausibel befüllt sein. Grenzwerte (z.B. negative Werte, null Drohnen) sollen ebenfalls validiert werden.

Abhängigkeiten:

Keine

4 Glossar

GUI Abkürzung für "Graphical User Interface"– grafische Benutzeroberfläche, die dem Nutzer das Steuern und Konfigurieren der Anwendung ohne Programmierkenntnisse ermöglicht.

Show-Konfigurationsmodul Softwarekomponente zur Erfassung, Validierung und Verwaltung von Drohnenshow-Parametern, inklusive Szenen, Bewegungen und Effekten.

Simulations-Engine Zentrale Komponente, die Drohnenbewegungen, Lichtverläufe und die 3D-Umgebung in Echtzeit oder annähernder Echtzeit berechnet und darstellt.

Projektmanagement-Modul Modul zur Verwaltung von Drohnenshow-Projekten, inklusive Speichern, Laden und Exportieren der Konfigurationsdaten.

Szenenprüfung Funktion zur automatischen Validierung von Benutzereingaben in einer Szene, insbesondere Prüfung auf fehlende Pflichtfelder und ungültige Werte.

Abnahmetest Testform, die prüft, ob ein Produkt den Anforderungen des Auftraggebers entspricht und für den produktiven Einsatz freigegeben werden kann.

Funktions-ID Eindeutige Kennung (z. B. F10, F20), die einer bestimmten Funktion im Pflichtenheft zugeordnet ist und in Testfällen zur Referenzierung dient.

Negativtest Gezielte Testmethode, bei der absichtlich ungültige oder unvollständige Eingaben gemacht werden, um die Fehlerbehandlung des Systems zu überprüfen.

Positivtest Testmethode mit korrekten Eingaben, bei der geprüft wird, ob das System wie erwartet reagiert und die Anforderungen erfüllt.

Unit-Test Isolierter Test einzelner Komponenten oder Methoden einer Software, typischerweise automatisiert mit einem Framework durchgeführt.

Framework Eine wiederverwendbare Softwaregrundlage, die bestimmte Funktionalitäten bereitstellt und als Gerüst für die Entwicklung eigener Anwendungen dient. Frameworks enthalten vordefinierte Komponenten, Konventionen und Erweiterungspunkte, um Entwicklungsaufwand zu reduzieren und die Wartbarkeit zu verbessern.

Integrationstest Testverfahren, bei dem mehrere bereits getestete Module zusammengeführt und deren Zusammenspiel überprüft werden.

Testumgebung Technische Infrastruktur (Hardware, Betriebssysteme, Software), auf der Tests durchgeführt werden, idealerweise identisch zur Zielumgebung des Produkts.

Performance-Analyse Untersuchung der Systemleistung unter verschiedenen Lastbedingungen, z. B. hinsichtlich Rendergeschwindigkeit oder Speicherverbrauch.

JSON „JavaScript Object Notation“ – textbasiertes Datenformat zur strukturierten Speicherung und zum Datenaustausch, auch für Projektexporte verwendet.

Fehlermeldung Systemrückmeldung, die beim Auftreten eines Problems in strukturierter und für den Nutzer verständlicher Form ausgegeben wird.

GitLab Webbasierte Plattform für Quellcode-Verwaltung, Projektmanagement und Continuous Integration/Deployment (CI/CD).

Simulation Die dynamische Nachbildung der geplanten Drohnenshow innerhalb einer virtuellen 3D-Umgebung basierend auf Benutzerkonfigurationen.

Timeline-Komponente Bestandteil der GUI, mit der Nutzer den zeitlichen Ablauf von Szenen, Formationswechseln und Lichteffekten steuern können.

Validierung Prüfung von Nutzereingaben oder Konfigurationsdaten auf Korrektheit und Vollständigkeit vor deren weiterer Verarbeitung.